

Redes de Comunicaciones: NanoFiles

Ibrahim Cherif Barry

ibrahim.c.b@um.es

49339889J

Grupo 1

Subgrupo 1.1

Arturo Trinidad Hoyos

ar.trinidadhoyos@um.es

24456325B

Grupo 1

Subgrupo 1.2

Convocatoria de mayo 2025

Tutor: José Rubén Titos Gil



Índice

1. Introducción -----	3
2. Protocolo de comunicación con el directorio -----	3
2.1. Formato de los mensajes -----	3
2.2. Autómatas -----	8
3. Protocolo de comunicación peer-to-peer -----	9
3.1. Formato de los mensajes -----	9
3.2. Autómatas -----	13
4. Mejoras implementadas -----	14
5. Comunicación NanoFiles ↔ Directorio en Wireshark -----	16
6. Conclusiones -----	17

1. Introducción

En este documento se especifica el diseño del protocolo de comunicación entre cliente NanoFiles y el servidor de Directorio, y el de los peers de NanoFiles entre sí.

La mejoras que hemos considerado a la hora de diseñar el protocolo son las siguientes: el comando *serve* con puerto efímero, el cual consiste en ampliar el comando *serve* para que el servidor de ficheros pueda utilizar cualquier puerto disponible (ephemeral port) para escuchar peticiones, en lugar de solamente el puerto 10000/TCP.

2. Protocolo de comunicación con el Directorio

En esta sección queda reflejado el protocolo de comunicación a seguir entre los clientes Nanofiles con el servidor de Directorio.

2.1. Formato de los mensajes

Para definir los mensajes NanoFiles ↔ Directorio, vamos a utilizar el formato “campo:valor”. El valor que tome el campo “operation” (código de operación) indicará el tipo de mensaje y por tanto su formato (qué campos vienen a continuación).

a) **Mensaje:** ping

Sentido de la comunicación: NanoFiles → Directorio

Descripción: El cliente NanoFiles trata de hacer un ping al directorio para comprobar que está accesible.

Ejemplo:

```
operation:ping\n\n
```

b) **Mensaje:** ping_welcome

Sentido de la comunicación: Directorio → NanoFiles

Descripción: El servidor de Directorio indica que está accesible.

Ejemplo:

```
operation:ping_welcome\n\n
```

c) **Mensaje:** register

Sentido de la comunicación: NanoFiles → Directorio

Descripción: El cliente NanoFiles indica que quiere compartir sus ficheros a otros peers de NanoFiles, y le pide al Directorio que los publique en su lista.

Ejemplo:

```
operation:register\n
serverport:1000\n
filename:file1.txt\n
filesize:4\n
filehash:a94a8fe5ccb19ba61c4c0873d391e987982fbbd3\n
filename:file2.pdf\n
filesize:8\n
filehash:2fd4e1c67a2d28fced849ee1bb76e7391b93eb12\n
filename:file3.zip\n
filesize:12\n
filehash:5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8\n
filename:file4.sh\n
filesize:16\n
filehash:da39a3ee5e6b4b0d3255bfef95601890afd80709\n
\n
```

d) **Mensaje:** register_ok

Sentido de la comunicación: Directorio → NanoFiles

Descripción: El servidor de Directorio le dice al NanoFiles que se han registrado sus ficheros y que otros peers NanoFiles sabrán que se está compartiendo.

Ejemplo:

```
operation:register_ok\n
\n
```

e) **Mensaje:** filelist

Sentido de la comunicación: NanoFiles → Directorio

Descripción: El cliente solicita al Directorio que le muestre la lista de ficheros compartidos por otros peers de NanoFiles.

Ejemplo:

```
operation:filelist\n\n
```

f) **Mensaje:** filelist_ok

Sentido de la comunicación: Directorio → NanoFiles

Descripción: El servidor de Directorio le enseña al Nanofiles la lista de ficheros publicados por otros peers NanoFiles.

Ejemplo:

```
operation:filelist_ok\nfilename:file1.txt\nfilesize:4\nfilehash:a94a8fe5ccb19ba61c4c0873d391e987982fbbd3\nfilename:file2.pdf\nfilesize:8\nfilehash:2fd4e1c67a2d28fced849ee1bb76e7391b93eb12\nfilename:file3.zip\nfilesize:12\nfilehash:5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8\nfilename:file4.sh\nfilesize:16\nfilehash:da39a3ee5e6b4b0d3255bfef95601890afd80709\n\n
```

g) **Mensaje:** filelist_empty

Sentido de la comunicación: Directorio → NanoFiles

Descripción: El servidor de Directorio le comunica al NanoFiles que no hay ningún fichero publicado ni que esté compartiendo ningún peer NanoFiles.

Ejemplo:

```
operation:filelist_empty\n\n
```

h) **Mensaje:** servers_sharing_file

Sentido de la comunicación: NanoFiles → Directorio

Descripción: el cliente NanoFiles pide al servidor de Directorio que, dada una subcadena, le muestre la direcciones IP de los servidores que comparten algún fichero con la subcadena que especificó.

Ejemplo:

```
operation:servers_sharing_file\nfilenamesubstring:down\n\n
```

i) **Mensaje:** servers_sharing_file_ok

Sentido de la comunicación: Directorio → NanoFiles

Descripción: el servidor de directorio le enseña a NanoFiles la lista de servidores que compartió algún fichero cuyo nombre contiene la subcadena que especificó.

Ejemplo:

```
operation:servers_sharing_file_ok\naddress:127.0.0.1:39074\naddress:127.0.0.1:23678\n\n
```

j) **Mensaje:** servers_sharing_file_empty

Sentido de la comunicación: Directorio → NanoFiles

Descripción: el servidor de directorio le enseña a NanoFiles que no hay ningún servidor que compartió algún fichero cuyo nombre contiene la subcadena que especificó.

Ejemplo:

```
operation:servers_sharing_file_empty\n\n
```

k) **Mensaje:** unregister

Sentido de la comunicación: NanoFiles → Directorio

Descripción: El cliente NanoFiles solicita al Directorio que quiere dejar de compartir sus ficheros a otros NanoFiles.

Ejemplo:

```
operation:unregister\nserverport:3087\n\n
```

l) **Mensaje:** unregister_ok

Sentido de la comunicación: Directorio → NanoFiles

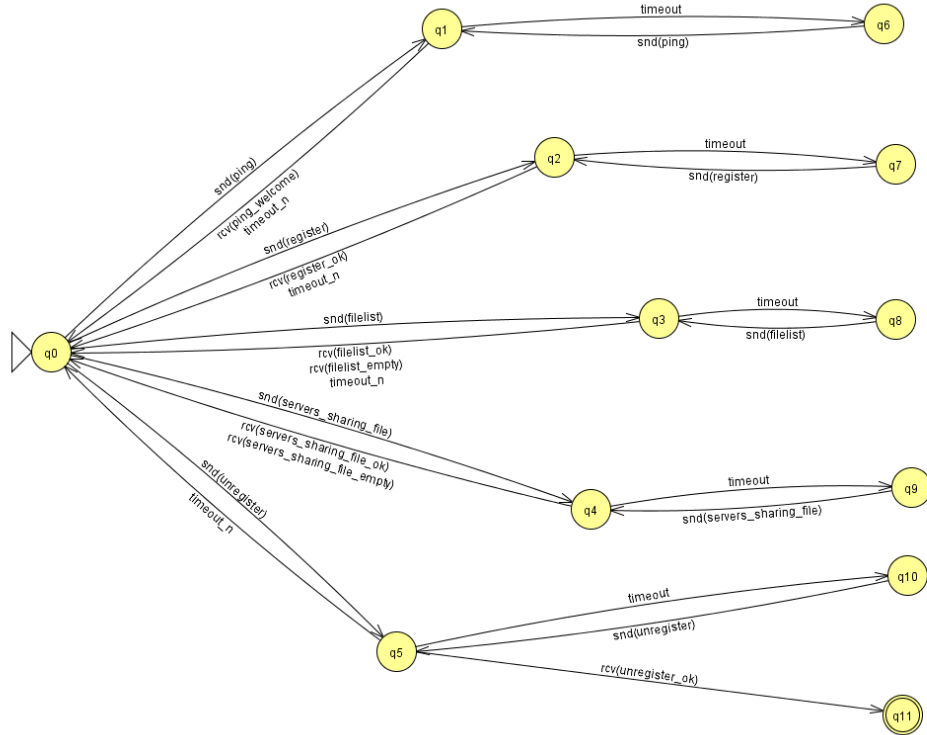
Descripción: el servidor de Directorio deja de compartir los ficheros del peer de NanoFiles que se lo solicitó

Ejemplo:

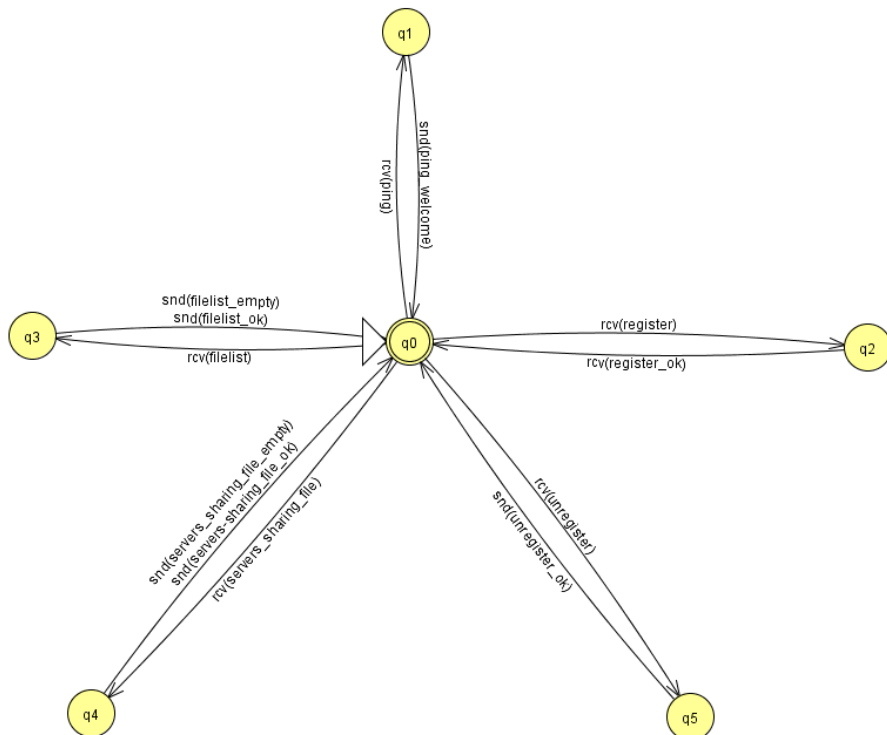
```
operation:unregister_ok\n\n
```

2.2. Autómatas

Nanofiles.



Directorio.



3. Protocolo de comunicación peer-to-peer

En esta sección queda reflejado el protocolo de comunicación que siguen los peers NanoFiles entre ellos.

3.1. Formato de los mensajes

Para definir los mensajes NanoFiles Cliente ↔ NanoFiles Servidor, vamos a utilizar mensajes binarios multiformato. El valor que tome el campo “opcode” (código de operación) indicará el tipo de mensaje y por tanto cuál es su formato, es decir, qué campos vienen a continuación.

a) **Mensaje:** FileNotFound (opcode = 1)

Sentido de la comunicación: NanoFiles Servidor → NanoFiles Cliente

Descripción: Este mensaje lo envía el peer servidor de ficheros al peer cliente de fichero para indicar que no es posible encontrar el fichero con la información proporcionada en el mensaje de petición de descarga.

Ejemplo:

Opcode (1 byte)
1

b) **Mensaje:** GetChunk (opcode = 2)

Sentido de la comunicación: NanoFiles Cliente → NanoFiles Servidor

Descripción: este mensaje lo envía el cliente para solicitar un fragmento (chunk) de un fichero. Contiene el desplazamiento (offset) desde el inicio del fichero, la cantidad de bytes a recibir, y el nombre del fichero.

Ejemplo:

Opcode (1 byte)	FileOffset (8 bytes)	ChunkSize (4 bytes)	FileName (n bytes)
2	32	16	“nombre.pdf”

- c) **Mensaje:** SendChunk (opcode = 3)

Sentido de la comunicación: NanoFiles Servidor → NanoFiles Cliente

Descripción: Este mensaje lo envía el servidor de ficheros al cliente en respuesta a un mensaje GetChunk. Incluye el fragmento (chunk) del fichero solicitado.

Ejemplo:

Opcode (1 byte)	FileOffset (8 bytes)	ChunkSize (4 bytes)	ChunkData (n bytes)
3	32	16	datos

- d) **Mensaje:** UploadFile (opcode = 4)

Sentido de la comunicación: NanoFiles Cliente → NanoFiles Servidor

Descripción: Este mensaje lo envía el cliente para solicitar la subida de un fichero. Contiene el nombre del fichero a subir.

Ejemplo:

Opcode (1 byte)	FileName (n bytes)
4	"nombre.txt"

- e) **Mensaje:** UploadAck (opcode = 5)

Sentido de la comunicación: NanoFiles Servidor → NanoFiles Cliente

Descripción: Este mensaje lo envía el servidor de ficheros al cliente para confirmar que la subida del fichero ha sido aceptada correctamente.

Ejemplo:

Opcode (1 byte)
5

- f) **Mensaje:** FileInfoRequest (opcode = 6)

Sentido de la comunicación: NanoFiles Cliente → NanoFiles Servidor

Descripción: el cliente NanoFiles le pide al servidor NanoFiles los metadatos del fichero que especifica como nombre.

Ejemplo:

Opcode (1 byte)	RequestedName (n bytes)
6	"name"

- g) **Mensaje:** FileInfoResponse (opcode = 7)

Sentido de la comunicación: NanoFiles Servidor → NanoFiles Cliente

Descripción: el peer servidor le entrega al peer cliente los metadatos del fichero solicitado por el peer cliente.

Ejemplo:

Opcode (1 byte)	FileName (n bytes)	FileSize (4 bytes)	FileHash (20 bytes)
7	"nombre"	8	7c222fb2927d82 8af22f592134e8 932480637c0d

- h) **Mensaje:** DownloadComplete (opcode = 8)

Sentido de la comunicación: NanoFiles Cliente → NanoFiles Servidor

Descripción: el peer cliente NanoFiles indica que la descarga se realizó satisfactoriamente.

Ejemplo:

Opcode (1 byte)
8

- i) **Mensaje:** ErrorMessage (opcode = 9)

Sentido de la comunicación: NanoFiles Cliente → NanoFiles Servidor

Descripción: el peer cliente NanoFiles indica que el fichero que recibió del peer servidor NanoFiles está corrupto.

Ejemplo:

Opcode (1 byte)
9

- j) **Mensaje:** FileAlreadyExists (opcode = A)

Sentido de la comunicación: NanoFiles Servidor → NanoFiles Cliente

Descripción: el peer servidor indica que el fichero el cual el cliente quiere subir, ya lo tiene.

Ejemplo:

Opcode (1 byte)
A

- k) **Mensaje:** UploadComplete (opcode = B)

Sentido de la comunicación: NanoFiles Cliente → NanoFiles Servidor

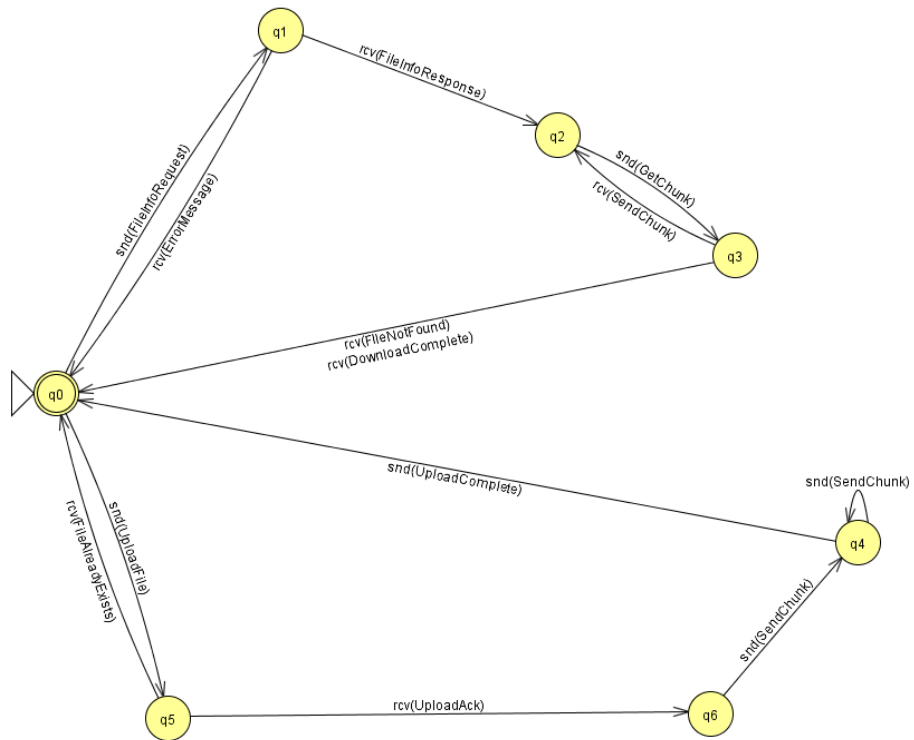
Descripción: el peer cliente de NanoFiles dice que ya ha hecho la subida completa del fichero.

Ejemplo:

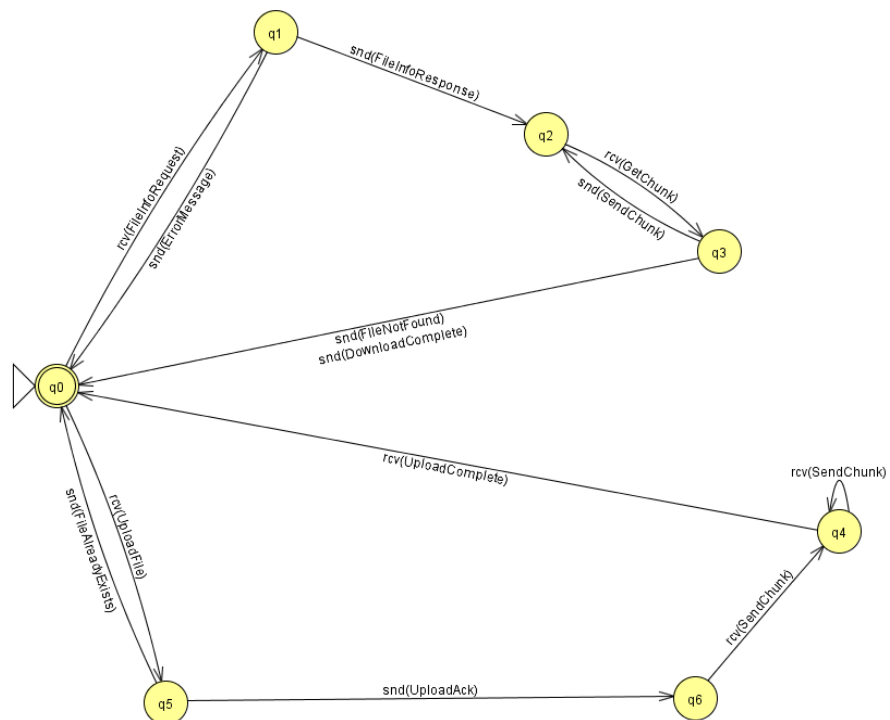
Opcode (1 byte)
B

3.2. Autómatas

Peer cliente de NanoFiles.



Peer servidor de NanoFiles.



4. Mejoras implementadas

Las mejoras que hemos considerado a la hora de diseñar el protocolo son las siguientes:

- **Comando `upload`:** se trata de subir uno de los ficheros compartidos por un peer a otro servidor, quien aceptará el fichero entrante siempre que no lo tenga ya entre sus ficheros disponibles.

He aquí una explicación: primero, el cliente envía al peer servidor de fichero un mensaje con el nombre del fichero a subir. El servidor revisa si el archivo ya existe: si es así, responde inmediatamente con `OPCODE_FILE_ALREADY_EXISTS` y la subida no continúa. Si el archivo no existe, responde con `OPCODE_UPLOAD_ACK`, indicando que está listo para recibir los datos. En ese caso, el peer cliente abre el fichero local y lo lee en fragmentos de 4096 bytes. Cada fragmento se envía al servidor como un mensaje `OPCODE_SEND_CHUNK`, hasta que todos los datos han sido enviados. Finalmente, el cliente notifica al servidor con el mensaje `OPCODE_UPLOAD_COMPLETE` para indicar que la transferencia ha terminado. Si ocurre un error en el proceso, se cierra la conexión.

Por un lado, la lógica del peer cliente se encuentra implementada en el método `uploadFile(FileInfo fileInfo)` de la clase `NFConnector`. Por otro lado, la lógica del peer servidor se implementó en el método `serveFilesToClient(Socket socket)` de la clase `NFServer`.

- **Comando `serve` con puerto efímero:** es una ampliación del comando `serve` para que el servidor de ficheros pueda utilizar cualquier puerto TCP disponible (ephemeral port) para escuchar peticiones, en lugar de solamente el puerto 10000/TCP.

La mecánica de la mejora es esta: el `NanoFiles` envía al Directorio un mensaje cuya operación es `register`. Uno de los atributos de ese `DirMessage` es el puerto TCP donde desea escuchar peticiones de los peers, además de la lista de los ficheros que pertenecen al peer que quiere registrarse como servidor. El Directorio comprueba que tiene ficheros. Si los tiene, envía un `DirMessage` de respuesta de tipo `register_ok`. En caso contrario, envía uno de tipo `register_denied`.

Si bien la lógica de `Nanofiles` la implementamos en el método `registerFileServer(int serverPort, FileInfo[] files)` de la clase `DirectoryConnector`, la lógica del Directorio la implementamos en

el método `sendResponse(DatagramPacket pkt)` de la clase `NFDirectoryServer`. Además, cabe destacar que en la clase `DirMessage`, hemos creado un método constructor con un atributo que representa el puerto donde escucharía peticiones el NanoFiles como peer servidor de ficheros.

- **Comando quit actualiza ficheros y servidores:** consiste en mantener permanentemente actualizada la información sobre ficheros compartidos (comando `filelist`). Si un peer que está sirviendo ficheros ejecuta la orden `quit`, deberá comunicarlo al directorio para dar de baja la lista de ficheros que comparte.

El procedimiento es el siguiente: el peer NanoFiles construye un mensaje de tipo `DirMessage` con una operación `unregister`, para enviárselo al Directorio. El Directorio, una vez comprueba que el tipo de la operación del mensaje recibido es `unregister`, busca en la lista de ficheros publicados aquellos que están asociados al puerto en el que está escuchando peticiones el peer NanoFiles. Si los hay, los elimina y envía al peer NanoFiles una respuesta de tipo `unregister_ok`. En caso contrario, envía una respuesta de tipo `unregister_denied`.

Mientras que la lógica del peer NanoFiles la hemos implementado en el método `unregisterFileServer(int serverPort)` de la clase `DirectoryConnector`, la lógica del Directorio la hemos implementado en el método `sendResponse(DatagramPacket pkt)` de la clase `NFDirectoryServer`.

5. Comunicación NanoFiles ↔ Directorio en Wireshark

Adjuntamos un escenario probado en una máquina a nivel local, donde se aprecia que el NanoFiles se identifica con la dirección 127.0.0.1:64661, y el Directorio se identifica con la dirección 127.0.0.1:6868.

Wireshark interface showing a packet capture of UDP traffic. The top bar indicates the capture is on the 'Ethernet 0' interface. The packet list shows 10 packets, all of which are UDP packets from 127.0.0.1 to 127.0.0.1. The packet details pane shows the selected packet (No. 10) with its structure: Ethernet II, Internet Protocol Version 4, and User Datagram Protocol. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
10	8.510017	127.0.0.1	127.0.0.1	UDP	48	64661 → 6868 Len=16
11	8.529976	127.0.0.1	127.0.0.1	UDP	56	6868 → 64661 Len=24
18	14.728409	127.0.0.1	127.0.0.1	UDP	52	64661 → 6868 Len=20
19	14.729489	127.0.0.1	127.0.0.1	UDP	58	6868 → 64661 Len=26
29	21.035767	127.0.0.1	127.0.0.1	UDP	406	64661 → 6868 Len=374
30	21.043327	127.0.0.1	127.0.0.1	UDP	55	6868 → 64661 Len=23
37	26.433229	127.0.0.1	127.0.0.1	UDP	52	64661 → 6868 Len=20
38	26.434804	127.0.0.1	127.0.0.1	UDP	392	6868 → 64661 Len=360
59	43.659848	127.0.0.1	127.0.0.1	UDP	71	64661 → 6868 Len=39
60	43.663628	127.0.0.1	127.0.0.1	UDP	57	6868 → 64661 Len=25

Y en esta otra captura, podemos observar el flujo que se sigue en el intercambio de mensajes de tipo DirMessage probado. Lo que está en rojo son los mensajes que envía NanoFiles, mientras que los que están en azul son aquellos que envía el Directorio.

[illegible]

6. Conclusiones

Por un lado, esta práctica nos ha ayudado a tener un mayor grado de lógica de programación, y a trabajar con sistemas de software más complejos, en vez de trabajar con unos pocos ficheros de código fuente. Probablemente nos ha ayudado a conocer un poco más la realidad del desarrollo de software.

Por otro lado, tras haber desarrollado este sistema de transferencia de ficheros, nos hemos dado cuenta de que la red no es magia. Existen unos protocolos por detrás que aseguran el funcionamiento de la red. El hecho de poner en práctica el uso de UDP y TCP nos ha ayudado a comprender sus conceptos vistos en la teoría de la asignatura.